



Muhammad Nugraha, M.Eng



CONTENTS

- 01.** Memanggil dan Membuat View
- 02.** Intro Blade View
- 03.** Laravel MIX
- 04.** Laravel - Bootstrap

Membuat & Memanggil View

○ Menampilkan View dari Controller

- ❑ Untuk menampilkan view dari Controller kita buat halaman view di `/resource/views/`
- ❑ Misal kita buat file `coba.blade.php`

```
4 <head>
5     <title>Halaman View</title>
6 </head>
7
8 <body>
9     <h1>Halaman Percobaan</h1>
10 </body>
11
```

- ❑ Panggil view dari controller dengan cara:

```
1 public function index()
2 {
3     return view("coba");
4 }
```

○ Mengirim Data ke View (cara 1)

- ❑ Buat variable `$a` di controller `MhsController` method `index()` yang berisi kalimat sembarang.
- ❑ Kirim variable `$a` ke view `coba.blade.php` dengan cara berikut:

```
1 public function index()  
2 {  
3     $a = "Data yang akan di kirim";  
4     return view('coba', compact('a'));  
5 }
```

- ❑ Buat tampilan di view yang mencetak variable `$a` seperti berikut:

```
1 <body>  
2 <h1>Halaman Percobaan</h1>  
3 <?php  
4 echo $a;  
5 ?>  
6 </body>
```

php

atau

```
1 <body>  
2 <h1>Halaman Percobaan</h1>  
3 {{ $a }}  
4 </body>
```

blade

- ❑ Buat router pemanggilannya dengan perintah berikut:

```
Route::get('/tes', [MhsController::class, 'index']);
```

nama Controller

nama method

○ Mengirim Data ke View (Cara 2)

- ❑ Buat variable `$nama` di controller `MhsController` method `nama()` yang berisi nama bebas.
- ❑ Kirim variable `$nama` ke view `coba.blade.php` dengan cara berikut:

```
1 public function nama()  
2 {  
3     $nama = "Muhammad Nugraha";  
4     return view('coba', ['kirim_nama' => $nama]);  
5 }
```

nama variable yang akan dikirim

nama key yang akan di tangkap di view sebagai variabel

nama view

- ❑ Buat tampilan di view yang mencetak nilai yang dikirim key seperti berikut:

```
1 <body>  
2     <h1>Halaman Percobaan</h1>  
3     {{ $kirim_nama }}  
4 </body>
```

- ❑ Buat router pemanggilannya dengan perintah berikut:

```
Route::get('/tes', [MhsController::class, 'nama']);
```

○ Mengirim Data ke View (Cara 3)

- ❑ Sama saja seperti di cara 2, hanya nilai yang di kirim langsung tidak menggunakan variabel
- ❑ Kirim data yang berisi 'Muhammad Nugraha' ke view `coba.blade.php` :

```
1 public function nama()  
2 {  
3     return view('coba', ['kirim_nama' => 'Muhammad Nugraha']);  
4 }
```

data/nilai
yang akan
dikirim

nama key yang akan di
tangkap di view sebagai variabel

nama view

- ❑ Buat tampilan di view yang mencetak data yang dikirim oleh key:

```
1 <body>  
2     <h1>Halaman Percobaan</h1>  
3     {{ $kirim_nama }}  
4 </body>
```

Komentor pada Blade

❑ Syntax komentar pada blade:

```
{{ -- komentar di sini -- }}
```


@if dan @elseif pada Blade

❑ Contoh if elsif pada blade:

```
1  @if($showSidebar === "left")
2  {{-- munculkan sidebar kiri --}}
3  <div> SIDEBAR KIRI! </div>
4
5  @elseif($showSidebar === "right")
6  {{-- munculkan sidebar kanan --}}
7  <div> SIDEBAR KANAN! </div>
8
9  @else
10 {{-- jangan munculkan sidebar --}}
11 <div> TIDAK ADA SIDEBAR DITAMPILKAN </div>
12 @endif
```

- ❑ Penulisan `if` ada blade tidak perlu kurung kurawal di setiap block kode statement.
- ❑ kode yang diawali dengan “@” seperti `@if`, `@endif` dll disebut dengan “**directive**”.

@unless pada Blade

❑ @unless mirip dengan if, @unless akan mengeksekusi kode block setelahnya, kecuali criteria yang dijadikan parameter terpenuhi.

❑ Contoh:

```
1  @unless( $status == "keren" )  
2      Kamu belum keren  
3  @endunless
```

❑ Kode blade di atas akan selalu memunculkan “Kamu belum keren” di layar kecuali nilai \$status adalah “keren”.

❑ Akhiri blok kode @unless dengan @endunless.

@switch pada Blade

- ❑ Seperti pada PHP `@switch` digunakan untuk menuliskan kode kondisional dengan banyak pilihan tidak bertingkat.
- ❑ Contoh:

```
1  @switch($vote_status)
2      @case(0)
3          <div> Tidak Setuju </div>
4          @break
5
6      @case(1)
7          <div> Setuju </div>
8          @break
9
10     @default
11         <div> Anda belum melakukan voting </div>
12 @endswitch
```

@isset dan @empty pada Blade

- ❑ Gunakan @isset untuk mengecek apakah sebuah variabel telah didefinisikan dan tidak bernilai NULL.

```
1  @isset($productList)
2  // kode view untuk menampilkan daftar produk jika $productList telah
3  // didefinisikan dan tidak bernilai NULL
4  @endisset
```

- ❑ Gunakan @empty untuk mengecek apakah sebuah variabel bernilai kosong (empty).

```
1  @empty($productList)
2  // kode jika $productList empty
3  @endempty
```

@auth dan @guest pada Blade

- ❑ Gunakan `@auth` untuk mengecek apakah pengguna yang sedang mengakses sudah login ke aplikasi Laravel kita.
- ❑ Kode yang ada di dalam block `@auth` hanya akan ditampilkan jika user telah login ke aplikasi.

```
1  @auth
2      Selamat datang!
3  @endauth
```

- ❑ Gunakan `@guest` untuk mengecek apakah pengguna yang sedang mengakses tidak login ke aplikasi.
- ❑ Kode yang ada didalam block `@guest` hanya akan ditampilkan ke user yang belum login.

```
1  @guest
2      Hai, silahkan login terlebih dahulu jika sudah punya akun.
3  @endguest
```

@foreach pada Blade

- ❑ @foreach membutuhkan variabel bertipe iterable, misalnya array dan minimal alias untuk masing-masing item didalamnya.

```
1  @foreach($productList as $product)
2  // sekarang kita punya akses ke masing-masing produk sebagai $product
3  // di dalam $productList
4
5  // tampilkan masing-masing nama produk dari array $productList
6  {{ $product->name }}
7  @endforeach
```

@forelse pada Blade

- ❑ @forelse ini hampir mirip dengan @foreach, bedanya kita bisa menyisipkan @empty di tengah-tengah blok kode untuk menampilkan sesuatu jika ternyata arranya kosong.

```
1  @forelse($productList as $product)
2  // sekarang kita punya akses ke masing-masing produk sebagai $product
3  // di dalam $productList
4
5  // tampilkan masing-masing nama produk dari array $productList
6  {{ $product->name }}
7
8  @empty
9  // jika kosong
10 <div>Belum ada produk</div>
11
12 @endforelse
```

🕒 @for dan @while pada Blade

- ❑ @for digunakan untuk melakukan perulangan seperti halnya `for` pada PHP.

```
1  @for($i = 0; $i < 10 ; $++)  
2      // kita akses nilai $i di sini  
3      // nilai $i bernilai 1 sampai 10;  
4  @endfor
```

- ❑ Seperti halnya `for`, @while juga memiliki fungsi yang sama dengan `while` pada PHP biasa.

Loop Variabel pada Blade

- ❑ Ketika kita melakukan operasi perulangan (looping), Blade menyediakan variabel yang bisa diakses dalam setiap perulangan (iteration). Variabel tersebut antaralain:

Variable	Penjelasan
<code>\\$loop->index</code>	Index perulangan saat ini. Mulai dari 0
<code>\\$loop->iteration</code>	Perulangan seberapa saat ini. Mulai dari 1
<code>\\$loop->remaining</code>	Berapa perulangan lagi sampai perulangan berhenti
<code>\\$loop->count</code>	Berapa jumlah data yang dilakukan perulangan. Bernilai sama seperti panjang array.
<code>\\$loop->first</code>	Mengecek apakah ini perulangan pertama
<code>\\$loop->last</code>	Mengecek apakah ini perulangan terakhir
<code>\\$loop->depth</code>	Tingkat nested loop. Apabila merupakan nested loop.
<code>\\$loop->parent</code>	Ketika menggunakan nested loop. Akses parent loop.

○ Loop Variabel pada Blade

□ Contoh:

```
@foreach($items as $item)
    @if($loop->first)
        <p>Our first element of the array</p>
    @endif

    <p>{{ $loop->iteration . ' / ' . $loop->count }}</p>

    @if($loop->last)
        <p>Our last element of the array</p>
    @endif
@endforeach
```

@php pada Blade

- ❑ Suatu ketika, kita mungkin perlu menyisipkan kode PHP seperti biasa pada template blade. Untuk melakukannya, kita bisa menggunakan @php seperti ini

```
1 @php
2     //kode PHP di sini
3 @endphp
```

Blade Layout, Section, & Componen

Layout & Section

- ❑ Layout digunakan untuk membuat view master yang akan selalu ditampilkan oleh view-view child yang menggunakannya.
- ❑ Dalam sebuah layout kita bisa memberikan tempat-tempat yang bisa digunakan oleh child view. Tempat-tempat tersebut adalah section.
- ❑ Misalnya, dalam layout utama, kita definisikan section sidebar, main_content, dan footer. Selanjutnya, setiap child view yang menggunakan layout utama dapat menempatkan kode view di masing-masing section yang tersedia di layout utama.

Contoh: Layout & Section

- ❑ Pada layout view di bawah, setiap kode html akan digunakan oleh child view layout tanpa harus menuliskannya lagi. Dengan demikian kita tidak perlu mendefinisikan tag html, head, title, dll pada tiap-tiap view.
- ❑ Dari layout view di bawah dapat kita baca bahwa layout itu menyediakan section sidebar, title, dan content yang didefinisikan menggunakan directive `@yield` dan `@section`.

```
1  <html>
2    <head>
3      <title>App Name - @yield('title')</title>
4    </head>
5    <body>
6      @section('sidebar') This is the master sidebar. @show
7
8      <div class="container">
9        @yield('content')
10     </div>
11   </body>
12 </html>
```

@yield dan @section pada Blade

- ❑ @yield("nama_section") digunakan untuk mendeklarasikan konten yang akan digunakan di child view. Untuk mengisinya, child view akan menggunakan @section dengan parameter nama yang sama dengan @yield seperti ini @section("nama_section").
- ❑ @section digunakan untuk mendefinisikan sebuah section, juga bisa untuk mengisi section yang diharapkan oleh parent view / layout melalui @yield.

resources/views/layouts/app.blade.php

```
1 <html>
2 <head>
3 <title>App Name - @yield('title')</title>
4 </head>
5 <body>
6 @section( 'sidebar' ) Navbar dari Layout
7 <hr />
8 @show
9
10 <div class="container">
11 @yield('content')
12 </div>
13 </body>
14 </html>
```

resources/views/pages/about.blade.php

```
1 @extends("layouts.app")
2
3 @section("title")
4     Tentang Kami
5 @endsection
6
7 @section("sidebar")
8     @parent
9     Sidebar dari halaman tentang kami
10 @endsection
11
12 @section("content")
13     Kami adalah Fullstack Developer yang penuh passion untuk memecahkan
14     masalah yang Anda miliki melalui aplikasi yang kami bangun!
15 @endsection
```

@parent dan @extends pada Blade

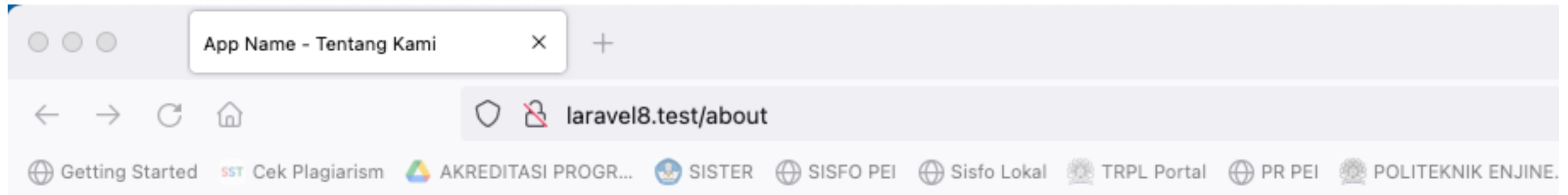
- ❑ Dalam child view kita bisa menampilkan juga konten yang ada pada parent dalam section tertentu, hal tersebut dilakukan dengan directive @parent.
- ❑ @extends digunakan pada setiap child view yang ingin menggunakan sebuah view sebagai parent / layout.

resources/views/layouts/app.blade.php

```
1 <html>
2   <head>
3     <title>App Name - @yield('title')</title>
4   </head>
5   <body>
6     @section( 'sidebar' ) Navbar dari Layout
7     <hr />
8     @show
9
10    <div class="container">
11      @yield('content')
12    </div>
13  </body>
14 </html>
```

resources/views/pages/about.blade.php

```
1 @extends("layouts.app")
2
3 @section("title")
4   Tentang Kami
5 @endsection
6
7 @section("sidebar")
8   @parent
9     Sidebar dari halaman tentang kami
10 @endsection
11
12 @section("content")
13   Kami adalah Fullstack Developer yang penuh passion untuk memecahkan
14   masalah yang Anda miliki melalui aplikasi yang kami bangun!
15 @endsection
```

Navbar dari Layout

Sidebar dari halaman tentang kami

Kami adalah Fullstack Developer yang penuh passion untuk memecahkan masalah yang Anda miliki melalui aplikasi yang kami bangun!

@yield vs @section

- ❑ Pada contoh sebelumnya terlihat penggunaan `@yield` dan `@section` yang serupa.
- ❑ Intinya untuk hal sederhana gunakan `@yield`, sedangkan untuk kebutuhan yang lebih advanced bisa digunakan `@section.. @show`.
- ❑ Karena semua yang bisa dilakukan dengan `@yield` bisa dilakukan dengan `@section.. @show`, namun tidak sebaliknya.

@include pada Blade

- ❑ Penggunaan `include` pada laravel sedikit berbeda dengan PHP biasanya.
- ❑ Penggunaan `include` pada Blade Laravel untuk memasukan file `blade.php` kedalam satu halaman, biasanya di gunakan untuk templating.
- ❑ Contoh, Buat 3 buah file berikut:
 - `head.blade.php`
 - `navbar.blade.php`
 - `footer.blade.php`

head.blade.php

```
1 <meta charset="utf-8">
2 <meta name="description" content="">
3 <meta name="author">
4
5 <title>Super Cool Layouts</title>
6
7 <!-- load bootstrap from a cdn -->
8 <link rel="stylesheet" href="//netdn
```

navbar.blade.php

```
1 <div class="navbar">
2     <div class="navbar-inner">
3         <a id="logo" href="/">Single Malt</a>
4         <ul class="nav">
5             <li><a href="/">Home</a></li>
6             <li><a href="/about">About</a></li>
7             <li><a href="/projects">Projects</a></li>
8             <li><a href="/contact">Contact</a></li>
9         </ul>
10    </div>
11 </div>
```

footer.blade.php

```
1 <div id="copyright text-right">© Copyright 2013 </div>
```

```
1 <!doctype html>
2 <html>
3 <head>
4     @include('includes.head')
5 </head>
6 <body>
7     <div class="container">
8
9         <header class="row">
10             @include('includes.header')
11         </header>
12
13         <div id="main" class="row">
14
15             @yield('content')
16
17         </div>
18
19         <footer class="row">
20             @include('includes.footer')
21         </footer>
22
23     </div>
24 </body>
25 </html>
```

Laravel MIX

(supplement)

Request CSS & JS

- ❑ Pada blade Laravel, cara untuk request css dan JS adalah seperti berikut:

CSS:

```
<link href="{{ url ('inspinia/font-awesome/css/font-awesome.css') }}"  
rel="stylesheet">
```

JS:

```
<script src="{{ url ('inspinia/js/bootstrap.js') }}"> </script>
```

- ❑ Bagaimana jika file CSS dan JS yang di panggil jumlahnya banyak?
- ❑ Agar simple gunakan Laravel MIX

Laravel MIX

- ❑ Untuk menggunakan Laravel Mix, harus sudah menginstall npm dulu di server / local kita.
- ❑ Jalankan command "npm install" untuk menginstall dependencynya terlebih dahulu.
- ❑ Jika sudah selesai, buka file **webpack.mix.js** yang ada di folder project Laravel.

▼ LARAVEL8

- > bootstrap
- > config
- > database
- > public
- > resources
- > routes
- > storage
- > tests
- > vendor
- .editorconfig
- .env
- .env.example
- .gitattributes
- .gitignore
- .styleci.yml
- artisan
- composer.json
- composer.lock
- package.json
- phpunit.xml
- README.md
- server.php
- JS webpack.mix.js**

```
1  const mix = require('laravel-mix');
2
3  /*
4   |-----
5   | Mix Asset Management
6   |-----
7   |
8   | Mix provides a clean, fluent API for defining some Webpack build steps
9   | for your Laravel applications. By default, we are compiling the CSS
10  | file for the application as well as bundling up all the JS files.
11  |
12  */
13
14  mix.js('resources/js/app.js', 'public/js')
15      .postCss('resources/css/app.css', 'public/css', [
16          //
17      ]);
18
```


Contoh: Laravel MIX

- ❑ File template dasar umumnya memanggil banyak file CSS/JS, seperti pada contoh di bawah ini:

```
<link href="{{ asset('template/css/bootstrap.min.css') }}" rel='stylesheet' type='text/css' />
<link href="{{ asset('template/css/style.css') }}" rel='stylesheet' type='text/css' />
<link href="{{ asset('template/css/font-awesome.css') }}" rel="stylesheet">
<link rel="stylesheet" href="{{ asset('template/css/icon-font.min.css') }}" type='text/css' />
<link href="{{ asset('template/css/animate.css') }}" rel="stylesheet" type="text/css" media="all">
```

- ❑ Pemanggilan CSS jika menggunakan Laravel MIX:

```
mix.styles([
    'public/template/css/bootstrap.min.css',
    'public/template/css/style.css',
    'public/template/css/font-awesome.css',
    'public/template/css/icon-font.min.css',
    'public/template/css/animate.css',
],
'public/css/hasil_combine.css') version();
```

→ Untuk memanggil semua file css nya

Contoh: Laravel MIX (1)

❑ Pemanggilan JS jika menggunakan Laravel MIX:

```
mix.scripts([  
    'public/template/js/jquery-1.10.2.min.js',  
    'public/template/js/Chart.js',  
    'public/template/js/wow.min.js',  
    'public/template/js/jquery.nicescroll.js',  
    'public/template/scripts.js',  
    'public/template/js/bootstrap.min.js',  
],  
    'public/js/hasil_combine.js').version();
```

Untuk memanggil semua file JS nya

❑ Pemanggilan hasil MIX:

```
<link href="{{ mix('css/hasil_combine.css') }}" rel="stylesheet">  
  
<script src="{{ mix('js/hasil_combine.js') }}"> </script>
```

Thanks !